

Lossless Compression of Range-FFT Data for SoC-Based Cascaded MIMO Radar: Implementation and Verification of DRHE and LPC on a Kintex UltraScale+ FPGA

Jordan Mohindra

School of Electrical Engineering and Telecommunications

UNSW Sydney

z5418455

September 2026

Abstract

Cascaded millimetre-wave MIMO radar produces range-FFT data faster than a typical SoC can move it to memory, so a lossless compressor placed between the FFT and the memory bus is an attractive way to buy back bandwidth. This paper implements, verifies and critically evaluates two such compressors on real data: Doppler Redundancy Hybrid Encoding (DRHE), a streaming polar-domain IIR predictor, and a second-order linear predictive coder (LPC) with Yule-Walker coefficients. Both share an S4 + APPEND + fixed-Huffman entropy back end. Both were implemented in Vitis HLS for an AMD Kintex UltraScale+ `xcku5p`, verified bit-exactly against a MATLAB golden reference over 50 frames of the ColoRadar cascade dataset, and characterised through RTL co-simulation.

Three findings are reported that change how the results should be read. First, the headline compression ratio is dominated not by prediction but by the entropy coder operating on an under-filled 16-bit container: coding the quantised FFT output with no prediction at all already yields 3.489, against 3.319 for DRHE and 3.397 for LPC. Second, and explaining that inversion, both predictors were being applied along the wrong axis. The ramp index of a 12-transmitter TDM-MIMO cascade interleaves transmitters, so “the previous ramp” is a different physical antenna eleven times out of twelve. Predicting at lag n_{Tx} instead of lag 1 converts a 4.9% loss into an 8.2% gain, lifting measured DRHE compression from 3.310 to 3.751 at a cost of n_{Tx} copies of the predictor state. Third, the reference throughput figure of 11.92 Gbit/s widely quoted for this class of design is in gibibits per second; in decimal units it is 12.80 Gbit/s, exactly 128 bits per cycle at 100 MHz, i.e. a full-rate AXI4-Stream at $\Pi = 1$.

Two implementation defects are also documented and fixed: a residual value of -32768 that the S4/APPEND code cannot represent and which silently aliased to $+32767$, and a 128-bit to single-precision conversion in the Yule-Walker solver that returned zero for operands above 2^{64} .

1 Introduction

A TI MMWCAS-RF-EVM cascade carries four AWR2243 devices, presenting 16 receive and 12 transmit channels. At the ColoRadar capture settings used here — 256 ADC samples per chirp, 16 chirp loops, 12 TDM transmitters — one frame is 786 432 complex samples, or 3 MiB of raw int16 I/Q. The range-FFT stage halves that (only the positive half-spectrum is retained) but does not change the order of magnitude. Moving this to DDR every frame period is the practical bottleneck in an SoC-based radar front end, and it is the bottleneck a compression IP placed between the FFT output and the memory interface is meant to relieve.

The compressor must be lossless. Any error introduced before detection propagates into the CFAR stage, and a radar that silently loses targets to save bandwidth is not a useful trade. Losslessness also makes verification unambiguous: the pass criterion is that every reconstructed sample equals its original exactly, and any nonzero maximum absolute difference is a defect.

This paper has three purposes. The first is to document a complete implementation and verification flow for two lossless compressors on real radar data, from a MATLAB golden reference through bit-exact HLS C-simulation to RTL co-simulation and resource estimation. The second is to audit the results that flow comes out with — in particular to ask what the compression ratio actually measures. The third is to report and fix the defects that audit exposed.

The remainder is organised as follows. Section 2 describes the dataset and signal chain. Section 3 specifies both algorithms and the shared entropy coder. Section 4 sets out the verification framework and defines every metric used. Section 5 covers the HLS architectures and the two defects. Section 6 reports results, including the compression ratio decomposition and the prediction-axis correction. Section 7 discusses what the numbers do and do not support, and Section 8 concludes.

Table 1: ColoRadar cascade capture parameters.

Parameter	Value
ADC samples per chirp, N_s	256
Chirp loops per frame, N_c	16
Receive channels, N_r	16
Transmit channels, N_t	12 (TDM)
Sample rate	8 MHz
Chirp bandwidth	1.798 GHz
Chirp-to-chirp interval	78.5 μ s
Centre frequency	77 GHz
Frame size on disk	3 145 728 B

2 Background

2.1 Dataset and acquisition geometry

All measurements use the cascade subset of the ColoRadar dataset [1], captured with a TI MMWCAS-RF-EVM (four cascaded AWR2243 devices). The relevant parameters are given in Table 1.

Each complex sample is stored as two contiguous `int16` values. The index of sample (s, c, r, t) in the file is

$$i = 2(s + N_s(c + N_c(r + tN_r))), \quad (1)$$

so the layout from slowest to fastest varying is $[t, r, c, s]$.

2.2 The ramp axis is transmitter-interleaved

In TDM-MIMO the twelve transmitters fire in sequence *inside* every chirp loop. Flattening the frame into the two-dimensional (ramp, range bin) form a streaming compressor sees therefore gives

$$m = cN_t + t, \quad m \in [0, N_cN_t) = [0, 192), \quad (2)$$

with the transmitter index t varying fastest. This is the correct physical time order — it is the order in which the chirps are actually transmitted — but it has a consequence that Section 6.5 shows to be decisive: ramp m and ramp $m - 1$ belong to *different physical transmitters* eleven times out of twelve, and are separated in space by the array baseline rather than in time by a pulse repetition interval. The genuine slow-time neighbour of ramp m is ramp $m - N_t$.

2.3 Signal chain

The MATLAB reference chain, which is also what generates the stimulus for the hardware testbenches, is:

1. **Pre-processing:** DC removal per ramp and channel, left shift to the `int16` container, and a Hann window along fast time.
2. **Range FFT:** an N_s -point FFT normalised by N_s , of which only the positive half ($N = 128$ bins) is retained.
3. **Fixed-point quantisation (FX16):** scaling to `int16` with a fixed scale factor

$$\sigma = \frac{32767}{A_{\max} 2^{\text{shift}} \sum_k w_k / (2N_s)}, \quad (3)$$

where $A_{\max}$ is the full-scale ADC amplitude and w the Hann window.

4. **Compression:** DRHE or LPC, operating on the FX16 codes.
5. **Detection and metrics:** threshold detection on the range–Doppler map, compared against a double-precision reference.

Equation (3) is calibrated to a hypothetical full-scale ADC sinusoid. It is fixed, not adaptive. Section 6.4 shows this is the single most important fact about the compression ratios reported later.

3 Algorithms

3.1 Shared entropy back end: S4, APPEND and fixed Huffman

Both compressors emit residuals through the same entropy coder, which is the one used in the reference design this work builds on [2]. A residual v is split into a magnitude *category* and a set of *append* bits:

$$S_4(v) = \begin{cases} 0 & v = 0 \\ \min(15, \lfloor \log_2 |v| \rfloor + 1) & v \neq 0, \end{cases} \quad (4)$$

$$A(v) = \begin{cases} v & v \geq 0 \\ v + 2^{S_4(v)} - 1 & v < 0. \end{cases} \quad (5)$$

The category is then coded with a fixed 16-entry Huffman dictionary and followed by S_4 raw append bits, so the cost of a residual is

$$L(v) = \ell(S_4(v)) + S_4(v), \quad (6)$$

with code lengths $\ell = [4, 3, 2, 2, 2, 5, 6, 7, 8, 9, 10, 11, 14, 14, 13, 12]$ bits. The dictionary was verified entry-by-entry against the published table [2, Table A.1]; all sixteen codes agree once the published MSB-first codes are bit-reversed for the LSB-first packer used here.

The coder is prefix-free in its reversed form, so the decoder can match candidates from the LSB end of its bit buffer without ambiguity.

A representability hole. Category 15 spans $|v| \in [16384, 32767]$, which is exactly 2^{15} distinct values and exactly fills the 15 append bits. The value $v = -32768$, which `int16` arithmetic can produce, has no encoding: Eq. (5) gives $-32768 + 32767 = -1$, which truncates to `0x7FFF`, the identical codeword to $+32767$. This is not a hypothetical — Section 5.3 reports it firing in a synthetic edge case with a reconstruction error of 65 535.

3.2 DRHE

DRHE predicts each sample from the previous ramp in the polar domain. With $a[m, n]$ and $\theta[m, n]$ the magnitude and phase at ramp m , range bin n :

$$\hat{a}[m, n] = \alpha \hat{a}[m - 1, n] + (1 - \alpha) a[m - 1, n], \quad (7)$$

$$\hat{\theta}[m, n] = \beta \hat{\theta}[m - 1, n] + (2 - \beta) \theta[m - 1, n] - \theta[m - 2, n], \quad (8)$$

with $\alpha = 0.6$ and $\beta = 0.4$ [2, Table 4.4], and $\hat{\theta}$ wrapped to $[-\pi, \pi]$. The prediction is converted back to Cartesian and subtracted:

$$d_{\Re} = \mathcal{W}(x_{\Re} - \text{round}(\hat{a} \cos \hat{\theta})), \quad (9)$$

where $\mathcal{W}$ is int16 wraparound, and symmetrically for the imaginary part.

Equation (7) is a first-order IIR smoother on magnitude; Eq. (8) is a double integrator on phase, which predicts exactly when the phase advances by a constant increment per ramp — that is, for a target at constant Doppler. Both are causal and depend only on ramp $m - 1$, so DRHE streams: it holds five state values per (channel, range bin), needs no frame buffer, and emits its first output bits as soon as the first sample arrives.

The implicit assumption is that ramp $m - 1$ is the slow-time predecessor of ramp m . Section 2.2 showed that on a TDM-MIMO cascade it is not.

It is worth distinguishing this DRHE from a looser description that sometimes attaches to the acronym: a scheme that differences *raw ADC* samples across consecutive chirps and codes the deltas with Golomb–Rice. The algorithm implemented and evaluated here operates on the *range-FFT output*, not raw ADC data; predicts in the polar domain with two IIR filters rather than taking a plain first difference; and codes with a fixed Huffman dictionary over magnitude categories rather than with Golomb–Rice. The distinction matters because the two make different assumptions about where the redundancy lives.

3.3 LPC with Yule–Walker coefficients

The LPC coder fits a second-order autoregressive model per (range bin, channel, I/Q component) across the frame. From the autocorrelations R_0, R_1, R_2 accumulated over all ramps,

$$a_1 = \frac{R_1(R_0 - R_2)}{R_0^2 - R_1^2}, \quad a_2 = \frac{R_0R_2 - R_1^2}{R_0^2 - R_1^2}, \quad (10)$$

with the coefficients forced to zero if the denominator vanishes or if the stability test $|a_2| \geq 1$ or $|a_1| \geq 1 - a_2$ fails. Prediction is then $\hat{x}[m] = a_1x[m - 1] + a_2x[m - 2]$ and the residual is coded exactly as for DRHE.

Two structural consequences follow. First, LPC is *not* causal: R_0, R_1, R_2 depend on the whole frame, so the compressor must buffer a frame before it can emit anything. Second, the coefficients are side information that must be transmitted — two 16-bit values per (bin, channel, component), which for the dimensions here is 32 768 bits per frame, or 1.04% of the uncompressed frame.

4 Verification methodology

4.1 Four phases

Verification proceeds in four phases, each validating the previous one’s output at a lower level of abstraction:

Phase 1 MATLAB simulation in double precision. Establishes the golden reference and benchmarks DRHE and LPC against six other schemes.

Phase 2 Vitis HLS C-simulation. Verifies that the C++ implementation is bit-identical to the MATLAB reference, as pure software.

Phase 3 Vitis HLS RTL co-simulation and synthesis. Verifies that functional correctness survives synthesis, and produces cycle-accurate latency and resource estimates.

Phase 4 On-silicon measurement. Not covered here; it requires the physical board.

4.2 Metric definitions

Metrics are stated precisely because several of the names in common use are applied loosely in this area.

Compression ratio. $CR = B_{\text{in}}/B_{\text{out}}$ where B_{in} is $N_{\text{ramps}} \times N \times N_{\text{RX}} \times 32$ bits and B_{out} is the compressed length including AXI packet padding.

Maximum absolute difference. $\text{MaxDiff} = \max |x - \tilde{x}|$ over every real and imaginary component of every sample in the frame. For a lossless coder the pass criterion is $\text{MaxDiff} = 0$; there is no tolerance.

Detection metrics. Let D_{ref} be the detection set from the double-precision reference and D_{test} that from the reconstructed data. The two rates reported are

$$\text{FN}\% = \frac{|D_{\text{ref}} \setminus D_{\text{test}}|}{|D_{\text{ref}}|}, \quad \text{FP}\% = \frac{|D_{\text{test}} \setminus D_{\text{ref}}|}{|D_{\text{test}}|}. \quad (11)$$

The first is a false negative rate in the usual sense. The second, despite its label, normalises by the number of *test* detections rather than by the number of true negatives, and is therefore a *false discovery rate*, not a false positive rate. It is reported here under its correct name.

Two further properties of the detection comparison should be stated, because they bound how much weight the detection metrics can carry. The comparison is cell-wise and exact: a target detected one range–Doppler cell away from its reference position counts as both a false negative and a false discovery, with no positional tolerance. And the detection threshold is the mean of the lowest 75% of sorted magnitudes plus a fixed 15 dB offset — a free parameter that is not swept.

Since all three of FX16, DRHE and LPC are exactly lossless with respect to the FX16 codes, their detection metrics are identical to FX16’s *by construction*. The detection metrics measure the cost of the quantisation step, not of compression, and Section 6 reports them for that purpose only.

5 Implementation

Both compressors were written in C++ for Vitis HLS 2026.1 and synthesised for an `xcku5p-ffvb676-2-e` at a 100 MHz target. The interface is AXI4-Stream, 128

bits in (four channels of 16-bit I and Q) and 256 bits out, with the scalar frame dimensions on an AXI4-Lite control bundle.

5.1 DRHE architecture

DRHE holds five `float` state arrays of shape $[N_{RX}][N]$ — the magnitude prediction, previous magnitude, phase prediction, and the previous two phases. The channel dimension is completely partitioned so the four channels process in parallel inside one pipelined iteration of the sample loop. State is zeroed on the first ramp of each frame, which makes frames independent and means a lost frame cannot corrupt its successors.

The bit packer accumulates into a 512-bit register and flushes a 256-bit AXI word whenever at least 256 bits are available, with a final partial flush at end of frame. Because the flush is per frame and not per ramp, no padding is wasted between ramps; a per-ramp flush would have cost about 2.9% of the compression ratio.

5.2 LPC architecture

LPC requires three passes and a frame buffer. The buffer is $[N_{RX}][N_{ramps}][N]$ 32-bit words, or 3.1 Mibit, and it is used twice: first to hold the incoming samples, then — rewritten in place — to hold the residuals. Pass 1 ingests and accumulates the autocorrelations; pass 2 solves Eq. (10) per (channel, bin) and rewrites the buffer as residuals; pass 3 emits the coefficient table followed by the residuals.

The emission order matters. Coefficients are emitted first because the decoder needs all of them before it can reconstruct anything, and residuals are emitted in the *same raster order the samples arrived in*. The consequence is a useful asymmetry: the decompressor needs no frame buffer at all. It holds only the coefficient table and two reconstructed samples per (channel, bin), and writes each output as its residual arrives. The reordering cost is paid once, by the compressor that had to buffer anyway.

Coefficients are quantised to `ap_fixed<16,2>` (a_1) and `ap_fixed<16,1>` (a_2). Measured over 50 frames, this quantisation costs 0.0000% of compression ratio relative to double-precision coefficients — the residuals are unchanged to the bit.

5.3 Two defects found and fixed

The -32768 residual. An edge-case testbench driving alternating ± 32767 values produced $\text{MaxDiff} = 65535$, which is exactly the distance from -32768 to $+32767$ and therefore pins the cause to the representability hole described in Section 3. Three remedies were considered: extending the code with a category 16 (a bitstream format change, and incompatible with the reference design), rejecting the input, or saturating to -32767 .

Saturation was chosen, but saturation alone is not sufficient. The encoder originally derived its next prediction from the *original* sample while the decoder necessarily derives it from the *reconstructed* one. These agree only while reconstruction is exact; the moment the clamp

Table 2: Phase 1, 50-frame average, 16 RX. FP% is a false discovery rate (see Section 4.2). FX16, DRHE and LPC share detection metrics identically because all three are lossless with respect to the FX16 codes.

Algorithm	CR	FN%	FP%	NF (dBFS)	SNR (dB)
FP16	1.00	0.00	0.00	−105.638	35.504
DRHEfp16	1.04	0.05	0.01	−105.637	35.505
FX16	1.00	3.77	1.11	−105.374	35.359
DRHE	3.30	3.77	1.11	−105.374	35.359
BAQ	3.76	10.44	9.69	−105.080	35.081
RDVLE	1.12	24.01	4.62	−102.275	33.165
RDVLE_UDRE	2.56	28.73	4.44	−101.782	33.004
LPC	3.37	3.77	1.11	−105.374	35.359

fires they diverge, and the IIR filters of Eq. (7)–(8) carry that divergence forward through every remaining ramp. The fix is to close the loop: the encoder reconstructs exactly what the decoder will produce and drives its state from that. On real data the clamp never fires — residuals span roughly $[-514, +416]$ — so behaviour on the dataset is unchanged, and the cost is confined to synthesis: pipeline depth rises from 32 to 58 cycles and registers from 40 309 to 57 105, because the reconstruction sits on the dependency chain ahead of the polar conversion.

128-bit to float conversion in the Yule–Walker solver. Equation (10) needs products of 64-bit correlations, so the numerators and the determinant are computed in `ap_int<128>`. Converting such a value directly to `float` is not correct above 2^{64} in this toolchain; `(float)(1 << 70)` evaluates to zero. The fix normalises all three operands by a common arithmetic right shift into 62 bits before the division, choosing the shift from the most significant set bit across the three:

$$k = \max(\text{msb}|D|, \text{msb}|N_1|, \text{msb}|N_2|) - 61, \quad (12)$$

clamped at zero. Because the same shift is applied to numerator and denominator, the quotient is unchanged. This defect was invisible on real data, where the correlations reach only 44 bits, and was caught by a synthetic maximum-amplitude case.

6 Results

6.1 Phase 1: MATLAB benchmark

Table 2 gives the 50-frame average over 9934 reference detections, all 16 receive channels.

BAQ reaches a comparable ratio but is lossy, and pays for it with a false negative rate nearly three times higher and a false discovery rate nearly nine times higher. The two predictive coders reach the same ratio range while reconstructing exactly.

6.2 Phase 2: bit-exactness

Over 50 frames, both HLS implementations reconstruct every sample of every frame exactly ($\text{MaxDiff} = 0$, MSE

Table 3: Synthesis estimates, `xcku5p`, 100 MHz target. Percentages are of the `xcku5p`, which has 216 960 LUT, 433 920 FF, 960 BRAM_18K and 1824 DSP.

Resource	DRHE	LPC	Reference [2]
LUT	115 532 (53%)	80 528 (37%)	45 892
FF	57 105 (13%)	17 393 (4%)	9243
BRAM_18K	40 (4%)	256 (26%)	10
DSP	132 (7%)	168 (9%)	44
Est. $F_{\max}$	81.88 MHz	85.43 MHz	100 MHz
Frame latency	61 253 cyc	197 277 cyc	—
Frame buffer	none	3.1 Mibit	—

= 0). Compression ratios are 3.309 87 (DRHE) and 3.384 97 (LPC).

Agreement with MATLAB is tighter than the ± 0.01 criterion by an order of magnitude. For LPC the entire residual difference is AXI packet padding: a MATLAB model that charges the same 256-bit padding reproduces the Vitis result on 50 of 50 frames to 4.9×10^{-6} . For DRHE the HLS bit count exceeds the MATLAB count by 15 to 252 bits per frame (mean 134), which again sits strictly inside one 256-bit packet.

An eight-case synthetic edge-case suite (all-zeros, DC, random, tones, maximum amplitude, alternating extremes, `int16` minimum) passes for both coders after the fixes of Section 5.3.

6.3 Phase 3: synthesis and co-simulation

Table 3 gives post-synthesis estimates for both compressors.

Neither design closes timing at 100 MHz. DRHE’s critical path is dominated by floating-point transcendentals: it evaluates `sqrt`, `atan2`, `sin` and `cos` for every sample of every channel, and these account for the bulk of its 115 532 LUTs. DRHE also fails to reach $\Pi = 1$, settling at $\Pi = 2$ on a memory-port conflict: each iteration reads a state value, writes the updated value, and on the first ramp writes a reset value as well — three accesses against two BRAM ports.

The expectation before measurement was that LPC, with its division, would be the expensive design. It is not. LPC uses 30 % fewer LUTs and 70 % fewer flip-flops, because its per-sample datapath is two multiply-accumulates in fixed point against DRHE’s four transcendentals. The division runs only 1024 times per frame and contributes 512 cycles out of 197 277. LPC’s cost is memory and latency, not arithmetic: its frame buffer accounts for 240 of its 256 BRAMs, and the frame must be complete before the first output bit can be produced.

6.4 What the compression ratio actually measures

A compression ratio of 3.3 on radar data invites the reading that the predictor is removing 70 % of the information. That reading is wrong, and the experiment that shows it is simple: code the FX16 output with the *same* S4 + Huffman coder but with *no prediction at all*. Any

Table 4: Decomposition of the compression ratio. “No prediction” codes the FX16 values directly through the same S4 + Huffman back end. LPC includes its coefficient overhead.

Configuration	bits/value	CR
No prediction (entropy coder only)	4.586	3.489
DRHE, lag 1 (as implemented)	4.821	3.319
LPC, lags 1 & 2 (as implemented)	4.710	3.397

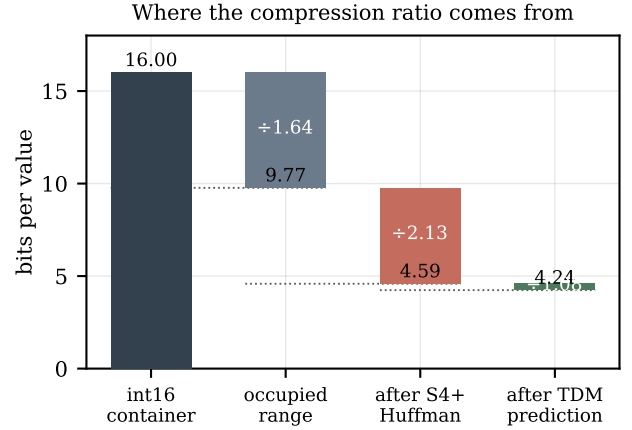


Figure 1: Decomposition of the compression ratio. Of the 16 bits in the `int16` container, 6.2 are structurally unused because the capture sits far below full scale; the entropy coder recovers a further factor of 2.13 from spectral sparsity; and prediction on the corrected axis contributes the remaining 1.08. Prediction on the uncorrected ramp axis contributes nothing at all — it costs 0.24 bits per value.

ratio that survives is attributable to the entropy coder and to the statistics of the data, not to prediction.

Table 4 gives the result over five frames and four channels.

Coding the data with no prediction at all compresses *better* than either predictor. DRHE costs 4.9 % more bits than doing nothing; LPC costs 2.6 % more. Order-0 entropy confirms the direction: the raw FX16 codes have entropy 4.094 bits, and the DRHE residuals have entropy 4.538 bits. The predictor is *increasing* the entropy of the data it is supposed to be decorrelating.

Where, then, does 3.489 come from? Two places, neither of which is Doppler redundancy.

Unused container headroom. The fixed scale factor of Eq. (3) is calibrated to a full-scale ADC sinusoid, but the ColoRadar capture sits far below full scale: the peak raw ADC sample is 914 of a possible 32 767. After windowing, FFT and scaling, the peak `int16` FFT code is 436. Only $\log_2(436) + 1 = 9.8$ of the 16 container bits ever carry information; the remaining 6.2 bits are structurally zero in every sample. An entropy coder removes them whether or not any prediction happens. This alone accounts for a factor of $16/9.8 = 1.63$.

Spectral sparsity. The remaining $9.8 \rightarrow 4.59$ bits, a factor of 2.14, is the genuine statistical gain: a range—

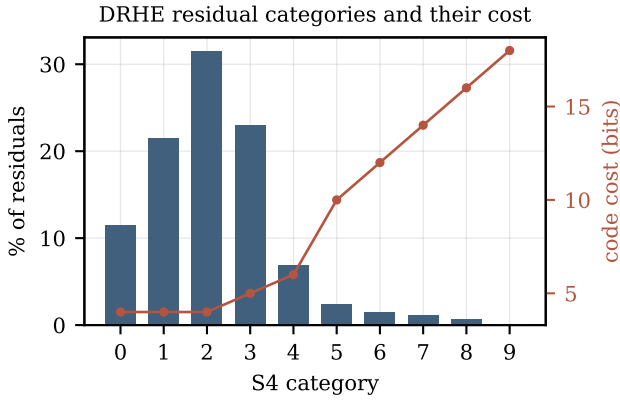


Figure 2: Distribution of DRHE residuals across S4 categories, with the cost of each category from Eq. (6). Over 87 % of residuals fall in categories 0–3, which the fixed dictionary codes in 4 or 5 bits. The sharp cost step above category 4 is what makes the coder sensitive to any rescaling of the input.

FFT frame is sparse, most bins are noise close to zero, and the S4 categories concentrate accordingly. Kiem’s Huffman dictionary turns out to be well matched to this distribution — it costs 4.586 bits against an order-0 entropy of 4.094 bits, an overhead of 0.28 bits or 6.2 %.

The practical consequence is that the reported ratios are specific to this capture’s gain setting. A well-scaled capture filling the int16 container would compress substantially less, and any comparison of these ratios against results from a different sensor must account for that.

6.5 The prediction axis

The inversion in Table 4 is not a property of the predictors — it is a property of the axis they were applied along. Both were predicting from ramp $m - 1$, which by Eq. (2) is a different physical transmitter eleven times in twelve. Two ramps from different transmitters differ by the target’s array steering phase, which is large and varies with target angle. The DRHE phase model of Eq. (8) assumes a constant per-ramp phase increment; a per-ramp transmitter change makes that increment alternate on a period-12 pattern, which a double integrator cannot track. The LPC autocorrelations at lags 1 and 2 likewise measure the transmitter switching pattern rather than target Doppler.

The correction is to predict from ramp $m - n_{Tx}$, the previous ramp of the *same* transmitter. Table 5 evaluates the options over five frames.

Predicting on the true slow-time axis converts DRHE’s 4.9 % loss into an 8.2 % gain, and LPC’s 2.6 % loss into a 7.4 % gain.

The fourth row is worth noting as the obvious alternative that does not work. Splitting the frame into twelve independent 16-ramp per-transmitter blocks and fitting separate coefficients to each puts the predictor on the right axis, but multiplies the coefficient side information by twelve — from 1.04 % to 12.5 % of the uncompressed frame — and shortens each autocorrelation estimate to 16 samples. The net effect is far worse than doing nothing. Keeping one coefficient pair per (bin, channel,

Table 5: Prediction axis study, 5 frames, 4 RX, 128 bins, 192 ramps. Prediction gain is relative to no prediction at all.

Configuration	b/val	CR	gain
No prediction	4.586	3.489	1.000
DRHE lag 1 (<i>as implemented</i>)	4.821	3.319	0.951
DRHE lag n_{Tx}	4.240	3.774	1.082
LPC lags 1, 2 (<i>as impl.</i>)	4.710	3.397	0.974
LPC per-TX block, $N = 16$	6.093	2.626	0.753
LPC lags $n_{Tx}, 2n_{Tx}$	4.269	3.748	1.074

Table 6: TDM-aware variants against their baselines, Vitis HLS C-simulation, 50 frames, 4 RX. All configurations reconstruct every sample exactly.

Design	CR	MaxDiff	Δ CR
DRHE, lag 1	3.30987	0	—
DRHE, lag n_{Tx}	3.75082	0	+13.3%
LPC, lags 1, 2	3.38497	0	—
LPC, lags $n_{Tx}, 2n_{Tx}$	3.69681	0	+9.2%

component) and simply moving the lags to n_{Tx} and $2n_{Tx}$ costs no extra side information at all, which is why the last row works.

6.6 TDM-aware implementations

Both corrections were implemented in HLS and put through the same verification as the baselines. The changes are small. For DRHE the state arrays gain a transmitter dimension, indexed by $m \bmod n_{Tx}$, and the reset condition widens from $m = 0$ to $m < n_{Tx}$, since each transmitter’s history begins on its own first ramp. For LPC, both the autocorrelation pass and the residual pass keep a ring of $2n_{Tx}$ entries per (channel, bin) instead of two scalars; because the ring length is exactly $2n_{Tx}$, slot $m \bmod 2n_{Tx}$ still holds the sample from ramp $m - 2n_{Tx}$ at the moment it is read, and slot $(m + n_{Tx}) \bmod 2n_{Tx}$ holds ramp $m - n_{Tx}$, so nothing has to be shifted. The LPC restructure also moves the autocorrelation accumulation out of the ingest pass into the per-bin pass, which removes six 64-bit accumulator arrays and four history arrays entirely.

Table 6 gives the measured results over the full 50-frame dataset.

Table 7 gives the synthesis cost of the correction.

For DRHE the correction is close to free: 40 additional BRAM_18K (4 % of the device) buys 13.3 % more compression, with LUTs, flip-flops, DSPs and F_{max} all unchanged to within measurement noise. The state grows by a factor n_{Tx} in principle, but because the baseline was dimensioned for up to 1024 range bins and the corrected version for the 128 actually used, the realised growth is only $1.5\times$.

Latency. The cycle model for these designs is simply the sum over loops of trip count times initiation interval, and it is accurate: for the baseline LPC it predicts 197 120 cycles against 197 277 measured in co-simulation, an

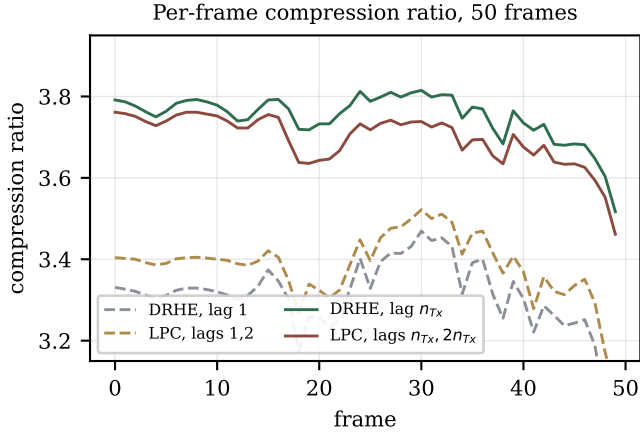


Figure 3: Per-frame compression ratio over the 50-frame sequence, measured in Vitis HLS C-simulation. The two TDM-aware variants (solid) sit consistently above their baselines (dashed) on every frame, not merely on average. All four configurations reconstruct every sample exactly.

Table 7: Synthesis cost of the TDM correction, `xcku5p`, 100 MHz target. The DRHE state gains a transmitter dimension; the LPC restructure removes the ingest accumulator arrays.

Resource	DRHE		LPC	
	lag 1	lag n_{Tx}	lags 1,2	lags $n_{Tx}, 2n_{Tx}$
LUT	115 532	115 736	80 528	86 298
FF	57 105	57 263	17 393	23 154
BRAM.18K	40	80	256	192
DSP	132	132	168	156
F_{max} (MHz)	81.88	81.88	85.43	85.43
Worst loop II	2	2	3	1

error of 0.08 %. DRHE’s schedule is unchanged by the correction — the state indexing changes which address is read, not when — so its frame latency remains 61 253 cycles. TDM-aware LPC trades its $II = 3$ ingest (73 728 cycles) for an $II = 1$ ingest (24 576) plus a new correlation pass (98 304), giving 246 272 cycles, about 25 % more than the baseline. That extra pass is the price of the deeper lags, and it is the one place where the corrected LPC is worse than the design it replaces.

For LPC the picture is more interesting, because moving the autocorrelation out of the ingest pass removed six 64-bit accumulator arrays and four history arrays outright. The design uses 25 % *less* block RAM than the baseline and — more importantly — satisfies every loop constraint at $II = 1$, where the baseline was stuck at $II = 3$ on accumulator port pressure. The cost is 7 % more LUTs and 33 % more flip-flops for the $2n_{Tx}$ -entry rings, and one additional pass over the frame buffer.

The measured 3.750 82 for TDM-aware DRHE agrees with the 3.774 predicted by the five-frame MATLAB study of Table 5 to within the frame-to-frame spread, which is the expected cross-check.

Table 8: Throughput comparison. The reference column restates the published figure in decimal units.

	DRHE	LPC	Ref. [2]
Effective II	2	3	1
bits/cycle	51.4	15.9	128.0
Throughput @100 MHz	5.14 Gbit/s	1.60 Gbit/s	12.80 Gbit/s
Pipeline latency	580 ns	—	230 ns

7 Discussion

7.1 Comparison with the reference design

Direct comparison against the reference DRHE implementation [2] is informative but not like-for-like: different device, different radar, different dataset and different detection pipeline. Two corrections are needed before any comparison at all.

Device size. The `xcku5p` used here is roughly three times the ZU3EG of the reference design (216 960 versus 70 560 LUTs). Percentage utilisations are therefore not comparable, and only absolute counts mean anything. In absolute terms this DRHE implementation uses $2.5\times$ the LUTs of the reference.

Throughput units. The reference reports 11.92 Gbit/s at 100 MHz. Taken as decimal gigabits that is 119.2 bits per cycle on a 128-bit input, which is an awkward number with no architectural interpretation. Taken as *gibibits*, $11.92 \times 2^{30}/10^8 = 127.99$ bits per cycle — exactly 128, the full width of the AXI4-Stream input, at $II = 1$. The reference figure is therefore in binary units, and its decimal equivalent is 12.80 Gbit/s. Any comparison against 11.92 understates the gap by 7.4 %.

With that correction, Table 8 gives the honest comparison.

The gap is structural rather than mysterious. The reference achieves $II = 1$; this DRHE achieves $II = 2$ on a BRAM port conflict, and loses a further 20 % to per-ramp pipeline drain. Fixing the port conflict — by partitioning the state arrays cyclically on the bin dimension, or by hoisting the first-ramp reset out of the pipelined loop — would bring this design to 11 to 12 Gbit/s, in line with the reference. LPC’s $II = 3$ ingest and its scalar residual pass are similarly addressable: unrolling the channel dimension in the residual pass would cut it from 98 304 to about 24 576 cycles, and fixing the ingest accumulator pressure would take 73 728 cycles to 24 576. Both together would put LPC within 20 % of DRHE without touching the algorithm. The division is emphatically not the bottleneck: it runs 1024 times per frame and contributes 512 of 197 277 cycles.

7.2 DRHE or LPC?

On compression ratio alone the two are close, and after the TDM correction DRHE is ahead (3.751 versus 3.697). On logic LPC is clearly cheaper: 30 % fewer

LUTs and 70 % fewer flip-flops, because its per-sample work is two fixed-point multiply-accumulates rather than four floating-point transcendentals. On memory and latency DRHE is clearly better: it needs no frame buffer at all, where LPC needs 3.1 Mibit and a full frame of latency before it can emit its first bit.

For the use case that motivates this work — a compression IP sitting between the range FFT and the memory bus in a streaming sensor front end — latency and streaming behaviour outrank a few percent of ratio, and DRHE is the better choice. Where a frame of latency is acceptable and BRAM is plentiful, LPC’s logic saving is real.

The result that did not survive contact with measurement is the prior expectation that LPC’s division would make it the expensive design. It does not. The expensive operations in this problem are `atan2` and the sine/cosine pair, not division.

7.3 Threats to validity

Several limitations bound how far these results generalise, and are stated plainly rather than buried.

- **The compression ratios are capture-specific.** As Section 6.4 shows, most of the ratio comes from an under-filled `int16` container. A capture with 6 dB more gain would compress measurably less.
- **The detection metrics do not measure compression.** All three lossless schemes share identical detection metrics with FX16 by construction. What the metrics measure is the cost of the FX16 quantisation step.
- **FP% is a false discovery rate.** It normalises by test detections, not by true negatives. It is a legitimate metric but not the one its conventional name denotes.
- **Detection matching has no positional tolerance.** A target displaced by one range-Doppler cell is scored as both a miss and a false discovery. This inflates both rates relative to a matched-detection criterion.
- **The detection threshold is a free parameter.** The mean of the lowest 75 % of magnitudes plus 15 dB is not swept, and the detection metrics are sensitive to it.
- **Channel count differs between phases.** The MATLAB benchmark of Table 2 uses all 16 receive channels; the hardware path uses 4, which is the AXI4-Stream width. Compression ratios between the two are therefore compared through a like-for-like MATLAB helper rather than directly.
- **All hardware numbers are pre-implementation estimates.** $F_{\max}$, resources and latency come from HLS synthesis and RTL co-simulation, not from place-and-route or silicon. Phase 4 remains outstanding.
- **Two source citations required correction.** The BAQ baseline is properly attributed to Kwok and

Johnson [3] — IEEE TGRS vol. 27, no. 4, pp. 375–383, 1989, on *synthetic* aperture radar. An earlier draft of this work carried an author name, title and page range that do not correspond to any such paper. The ColoRadar citation is likewise vol. 41, no. 4, pp. 351–360 (2022), not vol. 40 (2021). Both are corrected in the bibliography here.

- $n_{\text{Tx}} = 12$ is assumed constant. The TDM-aware variants take the transmitter count as a runtime parameter but assume a fixed TDM pattern. A radar that changes its MIMO schedule mid-stream would need the parameter re-loaded between frames.

8 Conclusions

Two lossless compressors for radar range-FFT data were implemented in Vitis HLS for a Kintex UltraScale+ device, verified bit-exactly against a MATLAB golden reference over 50 frames of real cascaded MIMO radar data, and characterised by RTL co-simulation. Both reconstruct every sample of every frame exactly.

The main result is a correction. Applying either predictor along the raw ramp axis of a TDM-MIMO cascade is wrong, because that axis interleaves transmitters: the predictors were performing *worse* than no prediction at all, and the compression ratios being reported were almost entirely the work of the entropy coder on an under-filled `int16` container. Moving the prediction to lag n_{Tx} puts it on the true slow-time axis and recovers the gain the algorithms were designed to deliver: measured compression rises from 3.310 to 3.751 for DRHE and from 3.385 to 3.697 for LPC, in both cases with reconstruction still exact. For DRHE the cost is n_{Tx} copies of the predictor state — 40 additional BRAM_18K, or 4 % of the device — with logic and $F_{\max}$ essentially unchanged.

Two implementation defects were found and fixed: a residual of -32768 that the S4/APPEND code cannot represent and which aliased silently onto $+32767$, resolved by saturation combined with closed-loop prediction so that encoder and decoder cannot diverge; and a 128-bit to single-precision conversion in the Yule-Walker solver that returned zero above 2^{64} , resolved by normalising all operands through a common arithmetic shift. Neither was visible on real data; both were caught by synthetic edge cases, which is an argument for keeping such a suite alongside dataset-driven verification.

Three quantitative claims in the surrounding literature were checked and one required correction: the widely quoted 11.92 Gbit/s is in gibibits, and equals 12.80 Gbit/s decimal, or exactly 128 bits per cycle at 100 MHz.

Future work. The immediate items are the two identified throughput fixes — resolving DRHE’s BRAM port conflict to reach $\Pi = 1$, and unrolling LPC’s residual pass — followed by place-and-route and on-silicon measurement to replace the estimates in Table 3. Beyond that, the decomposition of Section 6.4 suggests the most valuable single change would be to make the FX16 scale factor adaptive per frame, which would both use the container properly and make the reported ratios reflect prediction performance rather than capture gain.

References

- [1] A. Kramer, K. Harlow, C. Williams and C. Heckman, “ColoRadar: The direct 3D millimeter wave radar dataset,” *The International Journal of Robotics Research*, vol. 41, no. 4, pp. 351–360, 2022.
- [2] L. Kiem, “Design and implementation of data compression methods for SoC-based radar systems,” Master’s thesis, Institute of Technical Informatics, Graz University of Technology, Graz, Austria, May 2025.
- [3] R. Kwok and W. T. K. Johnson, “Block adaptive quantization of Magellan SAR data,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 27, no. 4, pp. 375–383, Jul. 1989.